

Base de conocimiento vectorial para el análisis de fenómenos aeroespaciales y territoriales

CODEFEST AD ASTRA 2026 — Etapa 1 • Informe técnico Equipo: Los P-Valorados — Antonio (ingesta), Samuel (fragmentación), Lucas (codificación e índice), Pao (recuperación y evaluación)

1. Tesis y arquitectura

Este informe documenta la base de conocimiento vectorial construida para la Etapa 1: 1.826 documentos oficiales sobre tres fenómenos aeroespaciales y territoriales, 50 consultas en español y, por cada consulta, los 3 documentos y los 10 fragmentos más relevantes. La explicación natural ubica la calidad de un sistema así en el modelo: comparar encoders, subir dimensiones. La experiencia del proyecto sostiene otra cosa. La calidad se decidió en tres lugares que preceden al encoder: qué parte del corpus entra al índice, dónde se corta cada fragmento y con qué identificador se responde. Las tres decisiones hubo que corregirlas en auditoría, y las tres movieron las métricas más que cualquier elección de modelado. Las secciones 2 a 5 documentan el *pipeline* en su orden (ingesta, fragmentación, codificación, índice); la sección 6, la recuperación y las mediciones que fijaron cada constante; la 7, la reproducibilidad. Todas las cifras del texto provienen del validador automático de la entrega o del arnés de evaluación local (cálculo propio; detalle en `_audit/EXPERIMENTOS.md`).

La arquitectura final: 102.736 fragmentos de 1.737 documentos oficiales (de los 1.826 del índice de ADL), codificados con `intfloat/multilingual-e5-base` (768 dimensiones, prefijos `passage:` y `query:`), almacenados en un `IndexFlatIP` exacto con vectores a norma unitaria, y un módulo de recuperación híbrido (búsqueda densa fusionada con una señal léxica y reordenamiento final por relevancia; §6) que produce la salida en el esquema estricto de la Tabla 2. El grafo de conocimiento opcional no se implementó.

2. Corpus e ingesta (§2)

ADL entregó 1.826 documentos (~3,14 GB) en tres carpetas temáticas: 459 sobre IA y capacidades estratégicas (F1), 479 sobre seguridad del entorno espacial (F2) y 888 sobre dinámicas territoriales (F3), en siete formatos (PDF, JSON, CSV, XLSX, imágenes, TXT y teselas de mapa PBF). La ingesta recorre el árbol completo, asigna a cada archivo un identificador interno reproducible (SHA-1 de su ruta relativa) y extrae el texto según el formato: PyMuPDF para PDF, con OCR Tesseract (`spa+eng+por`, modelos `tessdata_best`) para los 46 PDF escaneados y para las imágenes; lectura estructurada para JSON; una línea `columna: valor | ...` por fila para CSV y XLSX; y decodificación de las capas vectoriales para PBF, donde cada municipio del mapa de Amazon Underworld se convierte en un registro textual de atributos. Las teselas de zoom 4 a 6 repiten, atributo por atributo, las de zoom 3; se verificó la identidad de los 1.409 registros en los cuatro niveles y se indexó una sola copia.

La auditoría del corpus empezó por la aritmética más simple: el inventario de la ingesta registraba 77 archivos para el F2 y el índice oficial de ADL lista 479. ¿Qué podría explicar que faltara el 84 % de un fenómeno completo sin que ningún proceso hubiera reportado error alguno? Parecería que el problema no estaba en la extracción, que procesaba sin fallos todo lo que encontraba, sino en lo que había para encontrar: la copia local del corpus había llegado truncada de la descarga, y la constante de verificación del propio *script* de ingesta (`EXPECTED_COUNTS: F2=78`, a un archivo del conteo real) delataba que nunca hubo más.

Un archivo ausente no lanza errores. Fuera del índice quedaban precisamente las colecciones a las que apuntan las 16 consultas de seguridad espacial, q017–q032 (CSIS Aerospace, SWF Counterspace, ESA Space Debris, UNOOSA). Se re-ingirió el fenómeno completo desde los archivos oficiales: 402 documentos nuevos, cero fallos de extracción, y el F2 pasó de 587 fragmentos a 42.918. En la evaluación posterior, las consultas de espacio recuperan exactamente esas fuentes.

Hubo un segundo defecto de ingesta. El extractor de JSON conservaba los campos prioritarios que encontrara y, si hallaba un título, no buscaba más: 100 documentos quedaron reducidos a menos del 35 % de su texto (los artículos de CEEEP, a un título de ~90 caracteres sin su resumen). El extractor corregido concatena los campos de cabecera (título, resumen, extracto); toma un solo campo de cuerpo, porque `body_text` y `body_paragraphs` resultaron copias idénticas en los artículos capturados de la web y concatenarlos duplicaba cada artículo; y cae al barrido completo de cadenas cuando la extracción estructurada cubre menos del 35 % del texto disponible. Se re-extrajeron los 964 JSON del corpus.

Fenómeno	Docs oficiales	Con fragmentos	Fragmentos	Formatos dominantes
F1 — IA y defensa	459	445	25.776	pdf, json, csv

Fenómeno	Docs oficiales	Con fragmentos	Fragmentos	Formatos dominantes
F2 — entorno espacial	479	474	42.918	pdf, json
F3 — territorial	888	818	34.042	json, pdf, pbf

Los 89 documentos oficiales sin fragmentos se explican por construcción: 70 son teselas PBF de zoom 4 a 6 (duplicados exactos de las de zoom 3, cuyo contenido sí está indexado); 13 son datasets bibliográficos de PubMed, ClinicalTrials y lit-covid, excluidos por decisión documentada, pues son listados crudos de identificadores de papers biomédicos que habrían aportado ~80 % del índice con valor de recuperación nulo para las 50 consultas; y 6 carecen de texto extraíble (5 fotografías sin contenido textual, verificadas a ojo, y un JSON vacío). La organización dispuso conservar la metadata de las imágenes “sin inventar contenido”. Así se hizo.

3. Fragmentación (§3)

La estrategia de fragmentación es híbrida y la dirige el formato. La Ruta A segmenta la prosa (PDF, JSON de artículos, TXT) en oraciones con PySBD y las agrupa hasta un presupuesto de ~256 tokens, el punto donde la literatura de recuperación densa ubica el equilibrio entre especificidad del vector y contexto suficiente (Wang et al., 2024; Karpukhin et al., 2020), con tope duro de 240 palabras por fragmento; la Ruta B trata cada fila de CSV o XLSX como unidad atómica, porque en datos tabulares la unidad mínima con sentido es el registro completo; la Ruta C hace lo propio con los atributos de cada municipio PBF. El solapamiento entre fragmentos consecutivos se evaluó y se dejó apagado: la evidencia publicada señala que duplicar contenido en las fronteras infla el índice sin mejorar la precisión de forma consistente (Shaukat et al., 2026; Smith y Troynikov, 2024).

El requisito de completitud lingüística (§3.3) merece una confesión metodológica: la primera versión del segmentador lo violaba en silencio. PySBD, con `clean=False`, trata cada salto de línea como fin de oración, y el texto extraído de PDF conserva la envoltura visual de sus líneas; un censo completo de fronteras midió el daño en auditoría: el 38,4 % de las fronteras entre fragmentos consecutivos de PDF caía a mitad de oración, y 1.777 de ellas a mitad de palabra (“state-of-the-” | “art results...”). La corrección cabe en dos líneas: des-guionar los cortes de línea y unir las líneas envueltas dentro de cada párrafo antes de segmentar. En los documentos de control, las fronteras defectuosas cayeron de 239 a 10 y de 438 a 21 (las residuales corresponden a títulos e índices sin puntuación); tras regenerar el corpus completo, el censo reporta 98,25 % de fronteras oracionales limpias en prosa, contra 61,6 % antes del arreglo.

Cada fragmento lleva los ocho campos obligatorios de la Tabla 1. `num_tokens` se calcula con el tokenizador real del encoder (XLM-RoBERTa, sin tokens especiales); solo 3 de los 102.736 fragmentos (0,003 %) exceden la ventana de 512 tokens una vez sumados el prefijo y los tokens especiales: tres oraciones patológicas cuyo vector pierde entre 1 y 4 tokens finales y cuyo texto de salida está íntegro.

4. Codificación semántica (§4)

El encoder es `intfloat/multilingual-e5-base` (HuggingFace, licencia MIT), evaluado contra los seis criterios del §4.3: (i) es multilingüe nativo, inicializado desde XLM-RoBERTa y entrenado contrastivamente en ~100 idiomas con representaciones alineadas, condición necesaria porque las consultas llegan en español y el corpus mezcla un inglés mayoritario con español y portugués; (ii) produce 768 dimensiones, el punto medio entre `e5-small` (384) y `e5-large` o `bge-m3` (1024), respaldado por la advertencia del propio §4.3 (“Dimensiones más altas no garantizan mejor rendimiento”); (iii) acepta 512 tokens de entrada, compatible con el presupuesto de fragmentación; (iv) su informe técnico (Wang et al., 2024) documenta su rendimiento en los *benchmarks* públicos de recuperación multilingüe, el criterio que descartó a `paraphrase-multilingual-mpnet`, optimizado para similitud de paráfrasis; (v) su licencia MIT está entre las preferidas por la especificación; y (vi) con 278 M de parámetros, codifica el corpus completo en cómputo modesto. El vector de cada fragmento es la media de los vectores de sus tokens (*mean pooling*) y se normaliza a norma unitaria antes de insertarse.

La familia E5 exige prefijos de rol: `passage:` al indexar, `query:` al consultar. Omitirlos degrada la recuperación de forma medible: en la verificación de integridad, el mismo texto codificado sin prefijo alcanza una similitud media de 0,976 contra su vector indexado, frente a 1,000000 con el prefijo correcto. La verificación de alineación (el punto donde una base vectorial puede estar rota sin dar señal alguna) se hizo re-codificando muestras estratificadas y comparándolas contra `index.reconstruct(i)`: similitud 1,000000 en todas las sondas, incluidas la primera y la última posición del índice. La línea *i* de `metadata.jsonl` corresponde exactamente al identificador interno *i* de FAISS, como dispone el §5.3.

5. Índice vectorial (§5)

El índice es un `IndexFlatIP` de FAISS (Johnson et al., 2019): búsqueda exacta por producto interno que, con vectores a norma unitaria, equivale matemáticamente a la similitud coseno que pide el §8.2. Para 102.736

vectores de 768 dimensiones (316 MB en float32), el índice plano responde en milisegundos y evita la pérdida de exactitud de las variantes aproximadas (IVF, HNSW), que el propio §5.2 reserva para corpus de millones de vectores. Aproximar aquí sería pagar precisión por una velocidad que nadie pidió. El índice se serializa con `faiss.write_index()` y se carga con `faiss.read_index()` sin dependencias adicionales.

6. Recuperación y salida (§8–§9)

El módulo de recuperación opera en cinco pasos deterministas, todos dentro de lo que la norma permite: el documento de Preguntas Frecuentes autoriza expresamente la búsqueda híbrida con BM25 (“pero debe existir la base de datos vectorial”) y el reordenamiento con cross-encoders (“La restricción aplica es para arquitecturas decoders”). Los pasos, en orden: (i) búsqueda densa de 1.000 candidatos, codificando la consulta con el mismo encoder del índice y el prefijo `query:`; (ii) fusión con un BM25 propio (la función clásica de puntuación léxica por frecuencia de término y rareza en el corpus; implementación determinista, sin dependencias) mediante Reciprocal Rank Fusion (Cormack et al., 2009), que puntúa cada candidato sumando $1/(k_0 + \text{posición})$ en cada lista ($k_0=60$, peso léxico 0,5), porque la señal léxica exacta recupera siglas y nombres propios (GAOR, RPO, Chocó) que el espacio semántico difumina; (iii) deduplicación de fragmentos casi idénticos por huella de texto normalizado, pues el texto de plantilla repetido entre ediciones de un mismo informe desperdiciaba posiciones del top-10; (iv) reordenamiento de los primeros 48 candidatos con el cross-encoder `BAAI/bge-reranker-v2-m3` (Chen et al., 2024; arquitectura XLM-RoBERTa, licencia Apache-2.0), que lee el par consulta–fragmento y emite un escalar de relevancia sin generar texto; y (v) salida de los 10 fragmentos más relevantes, con tope de 4 por documento para no gastar el *ranking* en un solo informe, y de los 3 documentos de mayor puntuación agregada, donde la agregación es el máximo por documento más 0,3 veces la media de sus dos siguientes mejores fragmentos, elegida sobre la suma pura porque el corpus mezcla artículos de un solo fragmento con atlas de más de mil y la suma favorece sistemáticamente al documento largo.

La salida se empareja contra el *ground truth* (el juicio oficial de relevancia con el que se calificará cada consulta) por el identificador que la organización dispuso. La FAQ aclara que la evaluación a nivel de documento usa el DOC_ID oficial del `Indice_Datos_Codefest.xlsx`: “Deben emparejar los resultados mediante ese DOC_ID oficial y conservar fuente como metadata”. En consecuencia, el `doc_id` de toda la base (metadata, índice y resultados) es el DOC_ID oficial de ADL (p. ej: `F2-SWF-023`); los 1.409 registros municipales derivados de teselas PBF se reagruparon bajo el DOC_ID de su tesela; y `fuentes` conserva la ruta relativa original del archivo. El mapeo es una función determinista, verificada contra las 1.826 filas del índice oficial. Cada línea de `resultados.jsonl` cumple el esquema de la Tabla 2 (50 líneas q001–q050, exactamente 3 documentos y 10 fragmentos, ≤250 palabras por fragmento, UTF-8 sin BOM), y un validador automático re-chequea estructura, cardinalidades, tipos, tope de palabras, alineación índice–metadata y fronteras oracionales.

Cada constante anterior sale de un arnés de evaluación local con dos pruebas. La primera, de tipo *known-item* (se conoce de antemano el documento del que salió cada consulta), construye 400 pseudo-consultas con las primeras 15 palabras de fragmentos estratificados por fenómeno y formato, y mide el cableado del sistema: el documento padre aparece en la terna el 89,5 % de las veces y el pool profundo lo captura entre los 100 primeros en el 99,5 %. La segunda corre las 50 consultas reales y compara configuraciones con *proxies* de consistencia temática (indicadores indirectos: la fracción del top-10 de fragmentos y de la terna de documentos que cae en el fenómeno de la consulta). Tres resultados fijaron el diseño: (i) la fusión con peso léxico 0,5 domina al denso puro (+5,5 puntos de documento en *known-item*) y también al peso 1,0, que en consultas reales gana consistencia de documentos (+2,0 puntos) pero pierde la de fragmentos (–2,2 puntos); es razonable pensar que la señal léxica fuerte arrastra texto de otro fenómeno que comparte vocabulario; (ii) anclar a la terna los documentos de los fragmentos mejor puntuados vale 10,5 puntos de precisión en la primera posición; (iii) el reordenamiento con cross-encoder no muestra costo alguno en los *proxies* y aumenta la diversidad de documentos del top-10 en 4,6 puntos. Cabe resaltar el límite de estas mediciones: sin el *ground truth* oficial, los *proxies* ordenan configuraciones entre sí, sin estimar las métricas oficiales en valor absoluto. El detalle completo está en el repositorio (`_audit/EXPERIMENTOS.md`).

7. Reproducibilidad

`generador.py` (Python ≥3.9; dependencias: `faiss-cpu`, `sentence-transformers`, `numpy`) lee el archivo de consultas (txt, jsonl, csv o pdf, con identificadores q001–q050), carga el índice y la metadata entregados, y escribe `resultados.jsonl` de forma determinista: sin semillas aleatorias que fijar, con desempates estables por identificador de fragmento y con el encoder anclado a una revisión concreta de HuggingFace. La reproducción se verificó en frío: un entorno virtual limpio, creado únicamente desde `requirements.txt`, sobre una copia fresca del directorio de entrega, regeneró un `resultados.jsonl` idéntico byte a byte al entregado (SHA-256 coincidente: `081163e5...5251`). El *pipeline* de construcción (ingesta, fragmentación, codificación, índice)

también está versionado y es re-ejecutable, incluido el *script* de indexación que originalmente vivía solo en un *notebook* de Colab.

La Etapa 1 deja, además, una lección de método que las cifras anteriores respaldan: las consultas del F2 recuperan sus fuentes porque un inventario contó 77 archivos donde debía haber 479; las fronteras de fragmento caen en oraciones completas porque un censo midió cuántas no lo hacían; y la terna es evaluable porque lleva el DOC_ID que el evaluador espera. Ninguna de las tres correcciones tocó el modelo. La calidad, en últimas, se jugó donde menos brilla: en contar, cortar y nombrar.

8. Referencias

- Wang, L. et al. (2024). *Multilingual E5 Text Embeddings: A Technical Report*. arXiv:2402.05672.
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. arXiv:2004.04906.
- Chen, J. et al. (2024). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings*. arXiv:2402.03216.
- Cormack, G. V., Clarke, C. L. A. y Büttcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods*. SIGIR.
- Johnson, J., Douze, M. y Jégou, H. (2019). *Billion-scale similarity search with GPUs* (FAISS). IEEE Transactions on Big Data.
- Shaukat, M. A., Adnan, M. y Kuhn, C. C. N. (2026). *A Systematic Investigation of Document Chunking Strategies and Embedding Sensitivity*. arXiv:2603.06976.
- Smith, B. y Troynikov, A. (2024). *Evaluating Chunking Strategies for Retrieval*. Chroma Technical Report.
- Especificación Técnica CODEFEST AD ASTRA 2026, Etapa 1 (v1.0, julio 2026) y documento oficial de Preguntas Frecuentes.